

Laboratorio Nro. 4

Comunicación en Sistemas Embebidos

Comisión Tauro - Aguirre

Actividad 1: Introducción a Raspberry Pi y Comunicación I2C

- Fijamos una IP estática tanto en la Raspberry como en nuestra computadora personal para poder establecer la comunicación por SSH. Desde Visual Studio Code logramos configurar un entorno que nos permite editar el código *master_receiver.cpp* de manera cómoda, eliminando la necesidad de utilizar un monitor u otros periféricos. En este entorno además de acceder a los archivos pudimos tener interacción con la consola y probar los comandos propuestos.
- Luego de establecer la conexión entre la Raspberry y el arduino, probamos el archivo de *master_receiver.cpp* en la Raspberry y el *slave_send.cpp* en el Arduino. Nos familiarizamos con este tipo de comunicación e hicimos pequeñas pruebas, como por ejemplo cambiar el tipo de mensaje enviado.

Actividad 2: Mediciones de luminosidad por I2C

Para esta actividad retomamos el desarrollo realizado en la Actividad 3 y añadimos los archivos necesarios para establecer el protocolo de comunicación entre la Raspberry Pi y el Arduino.

Sobre el laboratorio anterior, realizamos modificaciones en el *main.cpp* para inicializar la comunicación I2C utilizando la dirección 0x08, y además creamos el archivo *comm_protocol.cpp* (junto con su respectivo encabezado *.h*) que implementa toda la lógica necesaria para el intercambio de datos entre ambos dispositivos

En el *main.cpp* dentro del *setup* inicializamos el I2C como esclavo. En *comm_protocol.cpp* se encuentra la lógica implementada para el arduino y dejamos en la carpeta *deploy* el archivo “*master_receiver.cpp*” para que sea implementado desde la Raspberry.

Cabe mencionar que, además de lo mencionado antes, se realizaron modificaciones en la propia implementación del Laboratorio 3. En particular decidimos refactorizar algunas cuestiones del código que controla el ADC y los Timers. Un gran problema en el laboratorio anterior era que nuestra implementación hacía uso de dos timers por separado, utilizando hardware muy limitado de forma libre e innecesaria, por lo que decidimos pasar a utilizar un único timer con tics reducidos, de unos pocos milisegundos, lo que permite ahorrar recursos de hardware a la vez que la funcionalidad se mantiene correcta.

Protocolo de comunicación entre Arduino y Raspberry:

El protocolo de comunicación entre la Raspberry Pi (maestro) y el Arduino (esclavo) se basa en el envío de paquetes (frames) con una estructura bien definida. Cada paquete está formado por varios campos que permiten identificar los límites del mensaje, su tipo, su longitud y los datos que contiene.

El formato general del paquete es:

SOF	LEN	TYPE	PAYLOAD	EOF
0X7E	0X01	0X0#	VARIABLE	0X7F

Donde:

- **SOF (Start of Frame):** byte especial que marca el inicio del mensaje. En nuestro caso, vale 0x7E.
- **LEN (Length):** indica la cantidad de bytes totales del paquete, sin incluir el SOF.
- **TYPE:** define el tipo de mensaje que se está enviando. Donde:
 - 0x01: OBTENER_LUX
 - 0x02: OBTENER_MAX
 - 0x03: OBTENER_MIN
 - 0x04: OBTENER_PROM
 - 0x05: OBTENER_TODO
 - 0x81–0x85: respuestas correspondientes desde el Arduino.
- **PAYLOAD:** contiene los datos del mensaje, por ejemplo, el valor del LUX o varias mediciones. Puede ser de tamaño variable (incluso cero).
- **EOF_B (End of Frame):** byte especial que indica el fin del mensaje (0x7F). Se define con *_B* ya que se consideró que *EOF* puede estar reservado para otros usos.
- **ESC (Escape):** símbolo (0x7D) usado cuando un byte del payload coincide con alguno de los símbolos especiales (SOF, EOF o ESC). En esos casos, se envía el byte ESC.

De esta forma, por ejemplo, si la Raspberry envía el comando “OBTENER_LUX” el frame sería:

SOF	LEN	TYPE	EOF
0X7E	0X01	0X01	0X7F

Si el valor de LUX actual es por ejemplo 12.45, el arduino responde con algo como:

SOF	LEN	TYPE	PAYLOAD	EOF
0X7E	0X08	0X81	'1' '2' '.' '4' '5'	0X7F

- **0x7E** → Inicio del mensaje (SOF)
- **0x08** → Tamaño total = 8 bytes (1 de TYPE + 5 del payload + 1 del EOF + este mismo campo LEN)
- **0x81** → Tipo de mensaje (RESPONDER_LUX)
- **'1' '2' '.' '4' '5'** → Payload con el valor de luminosidad
- **0x7F** → Fin del mensaje (EOF)

Beneficios de indicar el comienzo y el fin de un mensaje y por qué es necesario un símbolo especial de escape:

En nuestro caso, usamos un protocolo propio entre la Raspberry Pi (maestro) y el Arduino (esclavo) donde definimos un símbolo de inicio (SOF = 0x7E) y un símbolo de fin (EOF = 0x7F) para cada mensaje.

De esta forma, cuando la Raspberry envía o recibe datos, puede detectar fácilmente dónde empieza y dónde termina cada paquete, incluso si los bytes llegan de forma continua por el bus I2C. Esto nos permite separar correctamente los mensajes sin depender de tiempos o pausas entre envíos.

Además, implementamos un símbolo de escape (ESC = 0x7D) para los casos en los que alguno de los datos dentro del payload coincide con 0x7E o 0x7F. En esos casos, el Arduino “escapa” el byte (lo precede con 0x7D) para que el receptor sepa que ese valor no representa un inicio o fin de mensaje, sino que forma parte del contenido del payload.

De esta forma, los diferentes caracteres especiales definidos en nuestra implementación son:

- **SOF (0x7E)** marca el comienzo del paquete.
- **EOF (0x7F)** marca el final.
- **ESC (0x7D)** permite enviar cualquier dato sin que se confunda con los símbolos de control.

Explicación archivo comm_protocol.cpp:

El flujo de comunicación comienza cuando la Raspberry Pi envía un comando I2C (como OBTENER_LUX, OBTENER_MAX, etc.) al Arduino. Al llegar los bytes, la función `comm_on_receive()` copia los datos en el buffer `rx_buf` y encola la función `comm_process()` mediante `fnqueue_add()`. Dentro de `comm_process()`, se analiza el tipo de mensaje con `parse_type()`, se obtiene el valor correspondiente de luminosidad (`lux_actual`, máximo, mínimo o promedio) y se construye un frame de respuesta en `response_buffer` usando `build_frame()`, aplicando los escapes necesarios y calculando la longitud total. Cuando la Raspberry solicita datos, se ejecuta `comm_on_request()`, que revisa si hay una respuesta lista y la envía vía I2C con `Wire.write()`. Finalmente, la Raspberry recibe el frame, parsea el tipo de mensaje y el payload, y muestra el valor de luminosidad correspondiente. Todo este flujo asegura que cada comando enviado tenga su respuesta correcta y que los símbolos de inicio, fin y escape sean manejados correctamente para mantener la integridad del mensaje.

Explicación de cada función

Función	Propósito	Funcionamiento
<code>build_frame_append(uint8_t *out, size_t &out_idx, uint8_t b)</code>	Agrega un byte al frame asegurando que los símbolos especiales (SOF, EOF_B, ESC) sean escapados.	Si <code>b</code> es SOF, EOF_B o ESC, primero agrega ESC y luego <code>b ^ 0x20</code> . Si no, agrega el byte directamente.
<code>build_frame(uint8_t *out, uint8_t type, const uint8_t *payload, size_t payload_len)</code>	Arma un frame completo listo para enviar por I2C.	Escribe SOF, reserva un byte para LEN, agrega TYPE con <code>build_frame_append</code> , agrega payload con <code>build_frame_append</code> , agrega EOF_B y calcula LEN real (tipo + payload). Retorna la cantidad total de bytes del frame.

<code>parse_type(const uint8_t *buf, size_t len)</code>	Determina el tipo de mensaje del frame recibido.	Verifica longitud mínima y que el primer byte sea SOF. Busca el primer byte "no escapado" después de LEN; si estaba escapado, aplica XOR 0x20 para recuperar el valor original.
<code>comm_process()</code>	Procesa el mensaje recibido en <code>rx_buf</code> y genera la respuesta correspondiente.	Llama a <code>parse_type</code> para obtener TYPE. Si TYPE = 0 → respuesta de error "ERR". Según TYPE, obtiene los valores de lux (<code>lux_actual</code> , mínimo, máximo, promedio), los convierte a ASCII (<code>dtostrf</code>) y arma la respuesta en <code>response_buffer</code> usando <code>build_frame</code> .
<code>comm_on_receive(int howMany)</code>	ISR que se ejecuta cuando la Raspberry envía datos.	Copia los bytes de I2C en <code>rx_buf</code> , guarda la longitud en <code>rx_len</code> y encola <code>comm_process</code> con <code>fnqueue_add</code> para procesarlo fuera del ISR.
<code>comm_on_request()</code>	ISR que se ejecuta cuando la Raspberry solicita datos.	Si <code>response_len > 0</code> → envía <code>response_buffer</code> por I2C y resetea <code>response_len</code> .
<code>comm_init()</code>	Inicializa la comunicación I2C y registra los callbacks.	Registra <code>comm_on_receive</code> y <code>comm_on_request</code> con <code>Wire.onReceive</code> y <code>Wire.onRequest</code> .

Explicación master_receiver.cpp:

El flujo general del programa comienza con la inicialización de la librería **pigpio**, necesaria para manejar el bus I2C, y la apertura de la comunicación con el **Arduino** en la dirección **0x08**.

Luego, el sistema espera que el usuario ingrese un comando (entre 1 y 5) que representa el tipo de medición que se desea obtener. Una vez recibido el comando, la función

send_command() construye y envía un paquete con el formato definido por el protocolo (SOF, LEN, TYPE, EOF). A continuación, la Raspberry utiliza **read_frame()** para leer la respuesta enviada por el Arduino, detectando el inicio y fin del frame, y manejando los posibles bytes escapados. Después, con **parse_payload()** se extrae el valor ASCII del payload (por ejemplo, la lectura de luminosidad “12.45”), y con **parse_type()** se determina qué tipo de respuesta se recibió (LUX, MAX, MIN, PROM o TODO). Finalmente, el programa muestra en consola el valor obtenido y vuelve a esperar un nuevo comando; si ocurren varios errores consecutivos de lectura, reabre automáticamente la conexión I2C. Cuando el usuario ingresa ‘q’, se cierra el canal I2C y se termina la ejecución del programa limpiamente.

Función / Sección	Propósito	Descripción resumida
gpioInitialise()	Inicializa la librería pigpio	Habilita el uso del bus I2C.
i2cOpen(1, 0x08, 0)	Abre la comunicación I2C	Conecta con el Arduino (dirección 0x08).
send_command(handle, cmd)	Envía un comando al Arduino	Crea y manda un paquete con SOF, LEN, TYPE y EOF.
read_frame(handle)	Lee el frame de respuesta	Espera datos, detecta inicio (SOF) y fin (EOF), y arma el vector de bytes.
parse_payload(frame)	Extrae el payload	Devuelve el valor ASCII de la luminosidad recibido.
parse_type(frame)	Determina el tipo de mensaje	Permite saber si la respuesta es LUX, MAX, MIN, PROM o TODO.
main()	Controla el flujo completo	Pide comando al usuario, envía, lee y muestra la respuesta; maneja errores y reintentos.

2) Usar un byte de **inicio (SOF)** y uno de **fin (EOF)** permite delimitar claramente cada trama, garantizando que el receptor identifique mensajes completos incluso ante errores o ruido en la línea. El **byte de escape (ESC)** evita confusiones si el contenido del mensaje incluye valores iguales a los de control, asegurando una decodificación correcta.

5) Si el Arduino no muestra datos, es más eficiente enviar los **ticks del ADC** en lugar del valor en lux. Transmitir datos crudos reduce el tamaño del mensaje, evita conversiones innecesarias a ASCII y mejora la velocidad y fiabilidad del enlace I2C.